CollabDoc: A Real-Time Collaborative Document Editor with Operational Transformation and Stylometric Authorship Attribution

Nitish Kumar
Reg. No. 24BCE1520
Vellore Institute of Technology, Chennai
Chennai, India

Balan N
Reg. No. 24BCE1594
Vellore Institute of Technology, Chennai
Chennai, India

***Abstract*—Group documents are routinely written by several people at once, yet the tools that let them edit together say little about who actually wrote what. This paper presents CollabDoc, a real-time collaborative document editor that couples a server-arbitrated operational transformation (OT) engine with an authorship-attribution engine. Edits converge under arbitrary concurrency; a run-length keystroke ledger records mechanical authorship; and a dependency-free softmax classifier over 44 scale-free stylometric features forms an independent opinion about the style of each block. The two sources of evidence are blended with a weight that depends on how much text a block offers, and a leave-one-out check flags passages that were typed by one collaborator but read like another's writing. The system runs as a single process or as a Kubernetes deployment in which a document-aware router pins each document to one pod and a Kafka broadcaster carries presence and notifications between pods. On a development machine the server acknowledged edits with a 2.3 ms median and 3.9 ms 95th-percentile round trip, against a 200 ms requirement; the OT engine converged on 2000 randomised concurrent-edit pairs; and the browser port of the engine agreed with the server implementation on every randomly generated case. We also describe the engineering decisions that shaped the design, including a teardown bug that silently dropped the last second of typing.**

***Index Terms*—operational transformation, collaborative editing, authorship attribution, stylometry, keystroke dynamics, WebSocket, Kafka, Kubernetes**

I. Introduction

Real-time co-editing has moved from research prototypes [1], [4] to an everyday tool. Its central difficulty is unchanged: when two users edit the same text concurrently, every replica must end in the same state without discarding either user's intention [2]. Two families of algorithms address this. Operational transformation (OT) rewrites each incoming operation against the operations that landed concurrently [1]–[3]; conflict-free replicated data types (CRDTs) design the data structure so that merges commute [7], [9]–[11].

Convergence is only half of what a group of authors needs. In coursework, research teams and editorial settings, the question that follows a shared draft is *who wrote this?* Authorship attribution has a long computational history [15]–[18], but it is normally applied to finished text and independently of the editor that produced it. A collaborative editor, however, observes something a stylometric model never sees: every keystroke and its owner.

CollabDoc combines the two. The editing core gives the system an authoritative, append-only operation log; the attribution engine uses that log both as evidence and as the source of training labels. The remainder of this paper states the problem, describes the architecture, and reports measurements taken from the implementation.

II. Problem Statement

We target a document server that satisfies the following requirements, derived from the project's software requirements specification (SRS):

1) **Convergence.** Any interleaving of concurrent edits leaves all clients and the server holding identical text.
2) **Latency.** A real-time update is acknowledged and fanned out within 200 ms.
3) **Attribution.** Each block of text is assigned to a collaborator with a calibrated confidence, and the system declines to answer when it lacks evidence.
4) **History.** Every version can be reconstructed and rolled back without rewriting the log.
5) **Scale-out.** The service runs on several backends without corrupting a document's state.
6) **Security.** Role-based access, safe rendering, and no disclosure of whether a document exists to a caller without access.

III. Highlights and Novelty

- **Ledger-supervised stylometry.** Text that the keystroke ledger proves a user typed becomes that user's labelled corpus, so the classifier learns from the mechanical record and then forms an opinion independent of it.
- **Length-aware blending.** Stylometric weight $\min(1, \ell/\ell_0)$ scaled by a 0.65 cap, so a four-word heading is decided by keystrokes and a full paragraph mostly by style, and the model can never fully overrule the recorded keystrokes.
- **Leave-one-out integrity report.** A passage is judged by a model retrained *without* it, removing the self-vouching

effect of copied text; the report also lists pastes and per-person effort.

- **Honest abstention.** With fewer than two authors with three samples each, the model returns "not trained" and attribution falls back to the ledger.
- **Dependency-free classifier.** About 200 lines of softmax regression; a fitted model is a few hundred floats and trains in milliseconds, so attribution runs inline rather than behind a model-serving hop.
- **Dual OT implementations, cross-checked.** Server (Python) and browser (JavaScript) engines are compared on random edits.
- **Single code path for scale-out.** All events leave through a broadcaster interface even in one process; a document-aware router keeps each document on one pod.

IV. Related Work

A. Collaborative editing

Ellis and Gibbs introduced dOPT and the idea of transforming concurrent operations [1]. The Jupiter system simplified the problem by routing all operations through a central server, so each client only needs to transform against one peer [4]; CollabDoc follows this server-centric model. Ressel et al. [5] and Sun et al. [2] formalised the convergence, causality and intention-preservation goals, and Sun and Ellis surveyed the algorithms [3]. Oster et al. showed that several early transformation functions violate convergence and proposed the TTF approach [6]. CRDTs [7] include Logoot [9], RGA [10], a JSON datatype [8] and the Yjs framework [11]; Attiya et al. give a specification and complexity bounds for collaborative text editing [13]. Differential synchronization [14] takes a diff-and-patch approach, and local-first software [12] motivates offline-capable designs. We chose OT because the system already has an authoritative server and because the operation log doubles as the version history.

B. Authorship attribution

Mosteller and Wallace's study of the *Federalist Papers* established function-word frequencies as stylistic evidence [15]. Surveys by Juola [18], Koppel et al. [17] and Stamatatos [16] organise later work, Burrows' Delta [19], [20] remains a strong baseline, and Narayanan et al. examine attribution at internet scale [21].

C. Keystroke dynamics

Keystroke timing is a well-studied biometric for authentication [22]–[24]. CollabDoc uses keystrokes differently: not their timing, but their *ownership* within a shared buffer.

V. System Architecture

Fig. 1 shows the request paths. REST handles everything that is not a keystroke (accounts, documents, sharing, history); a WebSocket [30] carries editing, presence and attribution updates. Both read through a per-document `DocumentSession`, so a REST read during an edit returns the live text. The server is FastAPI [36] with SQLAlchemy over SQLite or PostgreSQL; the client is plain JavaScript with no build step.

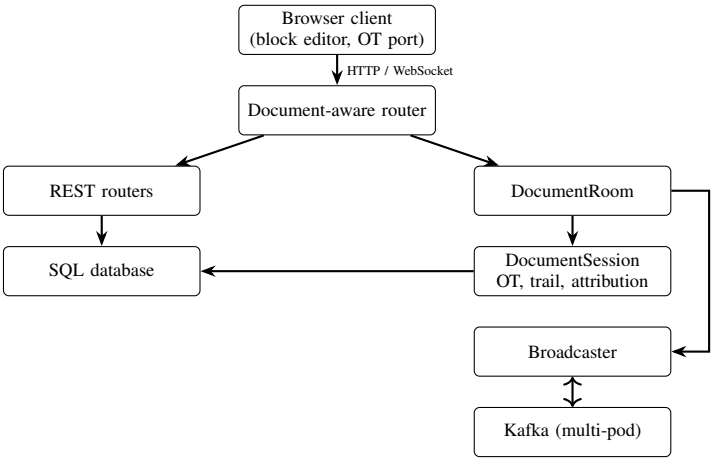


Fig. 1. Request paths. Every event, even in a single process, leaves through the broadcaster and returns through a subscription.

A. Data model

Seven tables map the hybrid model of the SRS through one ORM: users, documents, collaborators (OWNER/EDITOR/VIEWER), the operation log (one row per accepted operation), parsed blocks with attribution, periodic snapshots, and author samples. Block order uses a lexical rank, so concurrent inserts never renumber their neighbours. A rollback appends the inverse of the interval rather than rewriting history, so the log stays append-only and auditable.

VI. Operational Transformation Engine

An operation is a sequence of *retain*, *insert* and *delete* components over a plain-text buffer, serialised as a flat JSON array such as `[5, -2, "hi", 9]`. Builders keep operations canonical: adjacent retains merge, and an insert precedes a delete at the same position. Transformation must satisfy the convergence property

$$(a', b') = T(a, b) \;\Rightarrow\; \mathrm{ap}(\mathrm{ap}(d, a), b') = \mathrm{ap}(\mathrm{ap}(d, b), a'), \tag{1}$$

where ap applies an operation to a document d. Concurrent inserts at the same offset are tie-broken in favour of the first argument, so all replicas resolve ties identically.

Server rebasing. The server holds the text, a version counter and the operation log. A client sends {`version`, `operation`}; the server transforms it against every logged operation after that version, applies it, appends it, and broadcasts the transformed form. The sender receives an acknowledgement rather than an echo, which lets the client run the standard three-state machine (synchronised, awaiting confirmation, awaiting with buffer) with exactly one operation in flight. Because the server totally orders operations (the same idea of a single agreed order that underlies logical clocks [25]), each side only ever transforms against a single peer, the server,

which avoids the multi-site convergence conditions that made early peer-to-peer algorithms difficult [6].

Two implementations, one behaviour. The browser runs a port of the Python engine. A divergence would not raise an error; it would silently leave two editors with different text. A parity test therefore generates random concurrent edits in Python, transforms them in Node.js, and requires equal output component for component. Offsets travel as Unicode code points, because Python counts an emoji as one character and JavaScript as two.

Latency ordering. `submit` transforms and applies in memory, appends to the in-process log, acknowledges the sender and fans out to others. Only afterwards does a background saver drain the queue to the database (one-second cadence), and an attributor debounces 1.5 s after the last edit. At most one second of operations is therefore at risk, and the queue is drained synchronously on teardown.

Teardown bug. The first implementation flushed on disconnect with `await session.flush()` and lost data whenever a second editor was present. The ASGI server cancels the WebSocket handler the instant the socket drops, so every `await` in the `finally` block raised `CancelledError` before the write landed. The durable half of teardown now contains no suspension point (`flush_blocking`); a regression test covers it.

VII. Authorship Attribution

A. Two witnesses

Keystroke ledger. A run-length map from character offset to the user who typed it, advanced by the same operations the OT engine applies, so it cannot drift from the text. It is exact about *mechanical* authorship and blind to intellectual authorship: it cannot see that a paragraph was pasted.

Stylometric model. Forty-four scale-free features (sentence length, function-word habit, punctuation rates, contractions, vocabulary richness) are extracted from each block. Every feature is a rate or ratio, so a one-line comment and a three-paragraph section occupy the same feature space and the model learns style rather than length. A multinomial logistic regression trained by gradient descent is fitted per document on the ledger-labelled text of each collaborator. The implementation avoids a heavyweight dependency such as scikit-learn [35]; for 44 features and two to five classes, the fitted model is a few hundred floats and serialises into one database row.

B. Blending

For block b of length ℓ characters and training-sample size ℓ_0,

$$w_s = 0.65\,\min(1,\ \ell/\ell_0)\,, \qquad w_l = 1 - w_s, \tag{2}$$

$$\mathrm{score}(u) = w_s\,P_{\mathrm{style}}(u \mid b) + w_l\,\mathrm{share}_{\mathrm{ledger}}(u, b), \tag{3}$$

and the normalised winning score is stored as the block's confidence. Stylometry's vote fades as the block shrinks below one training sample, and its share is capped below one so a confident model cannot completely overrule the keystrokes.

C. Abstention

With fewer than two authors having three samples each, training returns `trained=False` with a reason, and attribution reverts to the ledger. The interface labels this "model: keystroke ledger only". A confident wrong answer is worse than an honest abstention.

D. Integrity report

The report summarises each person's share of the final text (ledger), effort (characters typed and deleted, active time, sessions; from the operation log), *pastes* (a single insert of 120 or more characters), and *style mismatches*. The live model is trained on what the ledger says each person typed, so a passage copied from Alice but typed by Bob sits in Bob's training data and vouches for itself. The report therefore judges each block with a model retrained without it, asking whether it reads like the typist's *other* writing. In a two-author demonstration this moved a copied passage from 59% (below the flag threshold) to 98% "reads like Alice". Flags direct a reviewer to passages to read; they are not proof of misconduct.

VIII. Scaling Out: Routing, Fan-out and Deployment

A document is live in exactly one process, because its version counter is held in memory and the operation log carries no unique `(document_id, version)` constraint; two processes numbering operations independently would corrupt the log silently. In the Kubernetes deployment, a Caddy router extracts the document id from `/ws/documents/{id}`, `/api/documents/{id}/...` and `/p/{id}` and applies rendezvous hashing [29] (a relative of consistent hashing [28]) over the stable pod names of a StatefulSet. Requests that name no document go to any pod. This is a deliberate point in the CAP trade-off space [26], [27]: a document is unavailable only while its pod fails over, but never inconsistent.

Presence, notifications and other events that must reach users connected to other pods travel over Kafka [31], one topic keyed by document id so a document's events keep their order on one partition. In the live AWS deployment (k3s, a lightweight Kubernetes [32], on one EC2 node in `ap-south-1`), three application pods, Kafka and the router run in the cluster; PostgreSQL runs on RDS, uploads on S3, and images in ECR. Infrastructure is described in Terraform and the stack costs about $2.5 per day while running. Every response carries an `X-Served-By` header to make the routing observable.

Security. Passwords use PBKDF2-HMAC-SHA256 with 260,000 rounds [33]; sessions are HS256 JSON Web Tokens [34]; every document route checks a role, and an unauthorised or unknown document returns 404 rather than 403, so the status code never confirms that an identifier is real. Login responses are identical for an unknown email and a wrong password; enabling link sharing mints a fresh token so revocation is real; and document text is HTML-escaped before inline markup is applied.

TABLE I
MEASURED RESULTS

Measure	Result
Edit round trip (8 ops, demo script)	2.3 ms median, 3.9 ms p95 (budget 200 ms)
Random concurrent edit pairs, convergence	2000 / 2000
Random compose / invert cases	1000 / 1000
Python vs. browser OT parity	all random cases equal
Held-out accuracy, synthetic corpora, 2 and 3 authors	100% (floor 85%)
Style vs. ledger agreement in demo	100% over 6 blocks
Automated tests collected (pytest)	205
Real-browser end-to-end checks	55

IX. Dataset

CollabDoc does not rely on an external corpus. The attribution model is trained at runtime on each document's own ledger-labelled text, and for evaluation we use three synthetic writing styles built to differ on the features the model measures (`server/tests/corpora.py`). These corpora are caricatures; they demonstrate that the pipeline is sound, not that real collaborators are this easy to separate. Because no original dataset was collected, no Zenodo DOI has been minted. Should a human-subject corpus be collected later, it would be archived as:

N. Kumar, B. N. and S. Vigneshwari, "CollabDoc collaborative-writing corpus," Zenodo, [DOI to be assigned].

X. Results

All figures below were measured on the authors' development machine with the in-memory broadcaster and SQLite; they are not cloud benchmarks.

Convergence and parity. The OT suite checks property (1) on 2000 random pairs, plus 1000 compose/invert cases, and the parity test requires the Node.js port to match the Python engine case for case.

Latency. The measured median of 2.3 ms is two orders of magnitude inside the 200 ms budget. The sample is small (eight operations from the scripted demo) and omits network distance, so it shows the server does not add meaningful delay rather than characterising wide-area behaviour. A multi-client load script exists but its results are not reported here.

Attribution. Accuracy of 100% on the synthetic corpora exceeds the 85% floor but should not be read as real-world accuracy. The more honest runtime measure is the *agreement rate* between stylometry and the ledger. In the demonstration, they agreed on all six blocks.

Defects found by testing. Real-browser testing found defects that the jsdom unit tests could not: a `hidden` attribute overridden by CSS `display` rules so the workspace overlaid the sign-in form; a silent parse error that left the page inert; a sidebar frozen at version 0; and a race in which two simultaneous openers could reset a room to version 0.

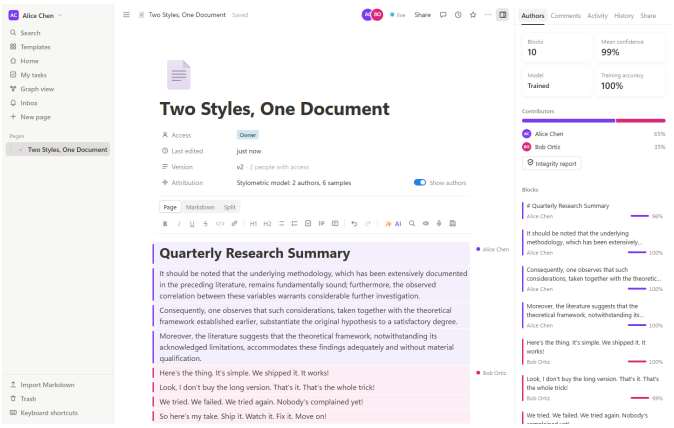


Fig. 2. Block editor with live collaborators and authorship attribution.

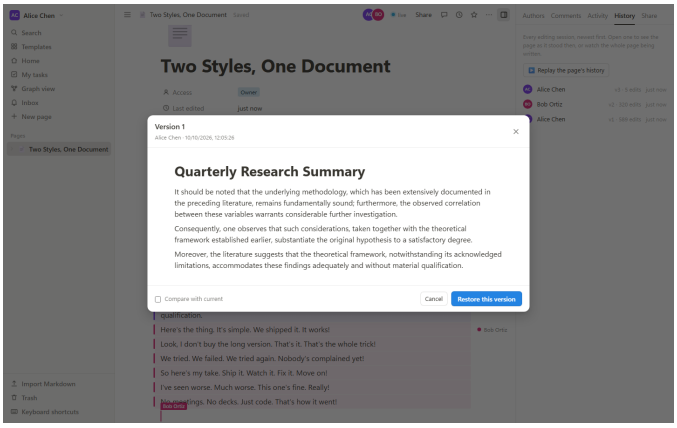


Fig. 3. Version history, drawn from the operation log.

A. Screenshots

Figs. 2–4 show the shared editor with presence, version history, and the integrity report.

XI. Limitations

Rich text is markup over a plain-text buffer, so formatting is a rendering concern and cannot span lines; styled-range OT or a CRDT is the upgrade path. Comments anchor by quoted text and lose their place if it is rewritten. Attribution needs writing to learn from, and style evidence between close collaborators is weak. Each document is pinned to one pod, so a pod failure moves its documents and editors should reload; a per-document lease, or a consensus-backed store such as one built on Raft [38], would remove this; the distributed-systems trade-offs involved are surveyed in [37]. The operation log grows without compaction. The measurements above come from a single machine and synthetic text, and a study with real collaborators is needed before attribution confidence can be treated as calibrated.

XII. Conclusion

CollabDoc shows that a server-arbitrated OT engine and an authorship model can share one source of truth: the operation

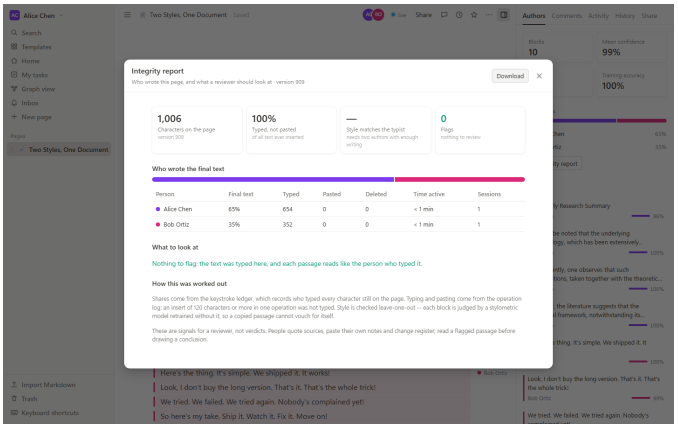


Fig. 4. Integrity report: per-person share and effort, pastes and style-mismatch flags.

log that makes editing convergent also supplies the labels, evidence and history that attribution needs. The design yields low acknowledgement latency, an explicit abstention rule, and an integrity report that corrects a failure specific to stylometry on shared text, namely that copied passages vouch for themselves. Future work includes real-user evaluation, styled-range synchronisation, log compaction, and per-document leases for fail-over.

Project Links

Project demo: not available; the AWS stack was torn down when the AWS credits were used up, and it can be recreated from the repository with Terraform.

GitHub repository: N. Kumar, B. N. and S. Vigneshwari, "CollabDoc: a real-time collaborative document editor with operational transformation and stylometric authorship attribution," https://github.com/Zerko-sd/CollabDoc.

Word-format copy of this paper: https://github.com/Zerko-sd/CollabDoc/blob/main/paper/collabdoc.docx.

Acknowledgment

The authors thank their faculty guide, S. Vigneshwari, for guidance on this work.

References

[1] C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," in *Proc. ACM SIGMOD*, 1989, pp. 399–407. doi:10.1145/67544.66963. https://doi.org/10.1145/67544.66963

[2] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen, "Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems," *ACM Trans. Comput.-Hum. Interact.*, vol. 5, no. 1, pp. 63–108, 1998. https://doi.org/10.1145/274444.274447

[3] C. Sun and C. Ellis, "Operational transformation in real-time group editors: issues, algorithms, and achievements," in *Proc. ACM CSCW*, 1998, pp. 59–68. https://doi.org/10.1145/289444.289469

[4] D. A. Nichols, P. Curtis, M. Dixon, and J. Lamping, "High-latency, low-bandwidth windowing in the Jupiter collaboration system," in *Proc. ACM UIST*, 1995, pp. 111–120. https://doi.org/10.1145/215585.215706

[5] M. Ressel, D. Nitsche-Ruhland, and R. Gunzenhäuser, "An integrating, transformation-oriented approach to concurrency control and undo in group editors," in *Proc. ACM CSCW*, 1996, pp. 288–297. https://doi.org/10.1145/240080.240305

[6] G. Oster, P. Urso, P. Molli, and A. Imine, "Data consistency for P2P collaborative editing," in *Proc. ACM CSCW*, 2006, pp. 259–268. https://doi.org/10.1145/1180875.1180916

[7] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Proc. SSS*, LNCS vol. 6976, 2011, pp. 386–400. https://doi.org/10.1007/978-3-642-24550-3_29

[8] M. Kleppmann and A. R. Beresford, "A conflict-free replicated JSON datatype," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 10, pp. 2733–2746, 2017. https://doi.org/10.1109/TPDS.2017.2697382

[9] S. Weiss, P. Urso, and P. Molli, "Logoot: a scalable optimistic replication algorithm for collaborative editing on P2P networks," in *Proc. IEEE ICDCS*, 2009, pp. 404–412. https://doi.org/10.1109/ICDCS.2009.75

[10] H.-G. Roh, M. Jeon, J.-S. Kim, and J. Lee, "Replicated abstract data types: building blocks for collaborative applications," *J. Parallel Distrib. Comput.*, vol. 71, no. 3, pp. 354–368, 2011. https://doi.org/10.1016/j.jpdc.2010.12.006

[11] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, "Near real-time peer-to-peer shared editing on extensible data types," in *Proc. ACM GROUP*, 2016, pp. 39–49. https://doi.org/10.1145/2957276.2957310

[12] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: you own your data, in spite of the cloud," in *Proc. ACM Onward!*, 2019, pp. 154–178. https://doi.org/10.1145/3359591.3359737

[13] H. Attiya, S. Burckhardt, A. Gotsman, A. Morrison, H. Yang, and M. Zawirski, "Specification and complexity of collaborative text editing," in *Proc. ACM PODC*, 2016, pp. 259–268. https://doi.org/10.1145/2933057.2933090

[14] N. Fraser, "Differential synchronization," in *Proc. ACM DocEng*, 2009. https://neil.fraser.name/writing/sync/

[15] F. Mosteller and D. L. Wallace, "Inference in an authorship problem," *J. Amer. Statist. Assoc.*, vol. 58, no. 302, pp. 275–309, 1963. https://doi.org/10.1080/01621459.1963.10500849

[16] E. Stamatatos, "A survey of modern authorship attribution methods," *J. Amer. Soc. Inf. Sci. Technol.*, vol. 60, no. 3, pp. 538–556, 2009. https://doi.org/10.1002/asi.21001

[17] M. Koppel, J. Schler, and S. Argamon, "Computational methods in authorship attribution," *J. Amer. Soc. Inf. Sci. Technol.*, vol. 60, no. 1, pp. 9–26, 2009. https://doi.org/10.1002/asi.20961

[18] P. Juola, "Authorship attribution," *Found. Trends Inf. Retr.*, vol. 1, no. 3, pp. 233–334, 2006. https://doi.org/10.1561/1500000005

[19] J. Burrows, "'Delta': a measure of stylistic difference and a guide to likely authorship," *Lit. Linguist. Comput.*, vol. 17, no. 3, pp. 267–287, 2002. https://doi.org/10.1093/llc/17.3.267

[20] S. Evert *et al.*, "Understanding and explaining Delta measures for authorship attribution," *Digit. Scholarsh. Humanit.*, vol. 32, suppl. 2, pp. ii4–ii16, 2017. https://doi.org/10.1093/llc/fqx023

[21] A. Narayanan *et al.*, "On the feasibility of internet-scale author identification," in *Proc. IEEE Symp. Security and Privacy*, 2012, pp. 300–314. https://doi.org/10.1109/SP.2012.46

[22] F. Monrose and A. D. Rubin, "Keystroke dynamics as a biometric for authentication," *Future Gener. Comput. Syst.*, vol. 16, no. 4, pp. 351–359, 2000. https://doi.org/10.1016/S0167-739X(99)00059-X

[23] K. S. Killourhy and R. A. Maxion, "Comparing anomaly-detection algorithms for keystroke dynamics," in *Proc. IEEE/IFIP DSN*, 2009, pp. 125–134. https://doi.org/10.1109/DSN.2009.5270346

[24] S. P. Banerjee and D. L. Woodard, "Biometric authentication and identification using keystroke dynamics: a survey," *J. Pattern Recognit. Res.*, vol. 7, no. 1, pp. 116–139, 2012. https://doi.org/10.13176/11.427

[25] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978. https://doi.org/10.1145/359545.359563

[26] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002. https://doi.org/10.1145/564585.564601

[27] E. A. Brewer, "Towards robust distributed systems," keynote, ACM PODC, 2000. https://people.eecs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf

[28] D. Karger *et al.*, "Consistent hashing and random trees," in *Proc. ACM STOC*, 1997, pp. 654–663. https://doi.org/10.1145/258533.258660

[29] D. G. Thaler and C. V. Ravishankar, "Using name-based mappings to increase hit rates," *IEEE/ACM Trans. Netw.*, vol. 6, no. 1, pp. 1–14, 1998. https://doi.org/10.1109/90.663936

[30] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, IETF, 2011. https://doi.org/10.17487/RFC6455

[31] J. Kreps, N. Narkhede, and J. Rao, "Kafka: a distributed messaging system for log processing," in *Proc. NetDB*, 2011. https://www.microsoft.com/en-us/research/wp-content/uploads/2017/09/Kafka.pdf

[32] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Commun. ACM*, vol. 59, no. 5, pp. 50–57, 2016. https://doi.org/10.1145/2890784

[33] K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: password-based cryptography specification version 2.1," RFC 8018, IETF, 2017. https://doi.org/10.17487/RFC8018

[34] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015. https://doi.org/10.17487/RFC7519

[35] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[36] S. Ramírez, "FastAPI documentation." https://fastapi.tiangolo.com

[37] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017. https://dataintensive.net

[38] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX ATC*, 2014, pp. 305–319. https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro